Prerequisites and objectives

- Prerequisites are Chapter 1's ADTs, invariants, and cost models, together with C++ object lifetimes.
- Define the Array ADT by its index domain. Derive multidimensional address formulas and explain the limits of contiguous storage.
- Trace sparse transpose and string matching. Distinguish characters from encoding units.

The Array ADT index contract

- A one-dimensional array maps a fixed index domain `{0,…,n-1}` to values. Both `get(i)` and `set(i,x)` require `0≤i<n`.
- Resizing is an extra capability of a dynamic sequence. It is not required by a fixed-length Array ADT.
- The postcondition of `set(i,x)` changes the value at i to x and preserves every other position.
- Contiguous memory is a representation, not the mathematical ADT definition. Segmented storage can offer indexed semantics with different costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C++ representation and lifetime

- `std::array<T,N>` has a compile-time fixed length. `std::vector<T>` can change length at runtime.
- Except for the `vector<bool>` specialization, vector elements occupy contiguous storage. `size()` counts existing elements. `capacity()` counts allocated element slots.
- `operator[]` is not a general bounds-checking mechanism. Examples check external indices before access.
- Reallocation invalidates existing element pointers, references, and iterators. `reserve` does not increase `size` to the requested capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Row-major two-dimensional addresses

- For an `R×C` array of fixed-width elements of w bytes, the element offset of zero-based `(i,j)` is `iC+j`.
- With base address b, the byte address is `b+w(iC+j)`. Apply this formula only to valid elements within the same contiguous allocation.
- For the original example `R=3,C=4,w=8`, `(2,1)` has element offset 9 and byte offset 72.
- Rows of `vector<vector<T>>` may be allocated separately. The entire matrix need not be one contiguous block, so the whole-matrix formula does not generally apply.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coordinates and addresses in row-major order

Count complete rows, then columns within the row; element and byte offsets have different units.

Row-major: from coordinates to contiguous storage

Matrix 3×4; select (2,1)

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	**(2,1)**	(2,2)	(2,3)

First skip 2 full rows: 2×4

Then advance 1 column: +1

offset = 2×4 + 1 = 9

Contiguous element slots (numbers are element offsets)

0	1	2	3	4	5	6	7	8	**9**	10	11

↑

Element width 8 bytes: byte offset 9×8=72; address b+72.

Heavy borders and arrow mark the target; the formula assumes one contiguous storage block.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Multidimensional strides: row-major and column-major

For dimensions $D_0 \times \cdots \times D_{d-1}$ and valid indices $0 \le i_t < D_t$, row-major changes the last index fastest. Advancing index t skips the product of the later dimensions:

$$\text{offset}_{row} = \sum_{t=0}^{d-1} i_t \prod_{u=t+1}^{d-1} D_u.$$

Column-major changes the first index fastest; its stride is the product of earlier dimensions:

$$\text{offset}_{col} = \sum_{t=0}^{d-1} i_t \prod_{u=0}^{t-1} D_u.$$

An empty product is 1. Multiply the element offset by width w to get the byte offset; check dimension products and index ranges before allocation or address arithmetic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Traversal order and cache behavior

- In row-major storage, increasing column index j in the inner loop reads adjacent elements.
- Increasing row index i in the inner loop instead gives a stride of C elements. It may reduce reuse of each cache line.
- Both traversals take $\Theta(RC)$ RAM operations. Actual times may differ. No cache speed ratio is claimed without measurement.
- Check multiplication bounds before allocating if $R \times C$ can overflow the index type. A valid logical shape does not guarantee a feasible allocation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

COO and CSR sparse representations

- `nnz=k` counts nonzero entries. COO stores triples `(row,column,value)` and separately retains the matrix dimensions.
- Our COO invariant requires row-major order, valid indices, unique coordinates, and no explicit zeros. Normalize other input by sorting, combining duplicate coordinates, and removing zeros.
- CSR stores `row_ptr`, `col_idx`, and `values`. Row i occupies `[row_ptr[i],row_ptr[i+1])`.
- CSR space is `O(R+k)`. The R term matters when many rows are empty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked sparse example

The matrix is 3×4. Entry IDs connect the nonzeros across representations. Row 1 has CSR interval $[1, 3)$, holding -2 and 7. For $x = [1, 2, 3, 4]^T$, $Ax = [10, 26, 8]^T$.

Sparse matrix: use entry IDs to connect COO and CSR

Matrix A

	0	1	2	3
0	0	#0 5	0	0
1	#1 -2	0	0	#2 7
2	0	#3 4	0	0

COO (row, col, value)

- #0 (0, 1, 5)
- #1 (1, 0, -2)
- #2 (1, 3, 7)
- #3 (2, 1, 4)

CSR: entry IDs match, even without color

	#0	#1	#2	#3
col_idx	1	0	3	1
values	5	-2	7	4

row_ptr = [0, 1, 3, 4]
row 1: [1,3) → #1, #2
#1: (1,0,−2) #2: (1,3,7)

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fast transpose counts columns

1. Count original entries in each column as `count[j]`.
2. Prefix-sum the counts to obtain starting positions `start[j]` for the transposed rows.
3. Scan original COO order. Write `(j,i,v)` at `next[j]` for entry `(i,j,v)`, then increment `next[j]`.
4. Preserve `start` as immutable interval boundaries. Do not overwrite it with the moving cursors.

Time is `O(C+k)`, auxiliary space is `O(C)`, and output space is `O(k)`. A complete output CSR also requires `C+1` row-pointer positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why fast transpose does not overwrite entries

- Column counts are `[1,2,0,1]`. Prefix starting positions are `[0,1,3,3]`.
- Column output intervals are disjoint, and each length equals its number of entries. Every input entry therefore has one unique output slot.
- Stable distribution in input row order keeps column indices increasing within each transposed row.
- The result is `(0,1,-2)`, `(1,0,5)`, `(1,2,4)`, `(3,1,7)`. An empty column naturally becomes an empty interval.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fast transpose: each output write

Follow the prefix boundaries and cursor for each write; an empty column occupies no output slot.

Fast transpose: count, partition, and stable scatter

count = [1, 2, 0, 1] start = [0, 1, 3, 3, 4]

next = [0, 1, 3, 3] (before scatter)

Input COO	Column	Output slot	Transposed item	next after write
(0,1,5)	1	1	(1,0,5)	[0,2,3,3]
(1,0,−2)	0	0	(0,1,−2)	[1,2,3,3]
(1,3,7)	3	3	(3,1,7)	[1,2,3,4]
(2,1,4)	1	2	(1,2,4)	[1,3,3,4]

Output in slot order 0 → 3:

(0,1,−2) (1,0,5) (1,2,4) (3,1,7)

Column 2 has count 0, so [3,3) is empty; original row order is preserved within each block.

start keeps the boundaries; next points to the next writable slot.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete COO transpose allocation and cursors

```
transpose(R,C,entries):
    require sorted unique valid nonzero entries
    count=[0]*C
    for (r,c,v) in entries: count[c]+=1
    start=[0]*(C+1)
    for c=0..C-1: start[c+1]=start[c]+count[c]
    next=copy(start[0:C]); out=array(len(entries))
    for (r,c,v) in entries:
        out[next[c]]=(c,r,v); next[c]+=1
    return (C,R,out)
```

For the same 3×4 example, (0,1,5) writes out[1], (1,0,−2) writes out[0], (1,3,7) writes out[3], and (2,1,4) writes out[2]. Start is [0,1,3,3,4]; final next=[1,3,3,4] equals the interval ends. Disjoint intervals with exactly matching write counts prevent overwrites and omissions. Stable distribution by original row preserves sorted columns within each transposed row.

Complete COO transpose allocation and cursors (continued)

Empty column 2 has interval [3,3). With R=0, C=4, retain result dimensions 4×0. Time is $\Theta(C+k)$, output space $\Theta(k)$, and auxiliary space $\Theta(C)$. Normalizing unsorted or duplicate input adds separate sorting/combination work. Sparse addition also requires equal dimensions, not merely equal nnz.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sparse arithmetic costs

- Add two sorted COO inputs by advancing as in merge sort. Combine equal coordinates and omit a sum of zero.
- For input nonzero counts k_1, k_2, the merge costs $O(k_1+k_2)$, excluding input normalization.
- CSR matrix-vector multiplication $y_i=\Sigma_j A_{ij} x_j$ costs $O(R+k)$, including output initialization.
- Sparse matrix multiplication is not generally $O(k_1+k_2)$. The output may become dense, and accumulating equal coordinates requires additional organization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete sparse addition and matrix-vector multiplication

```
add_COO(A,B):
    require same dimensions and canonical row-major entries
    i=0; j=0; out=[]
    while i<len(A) and j<len(B):
        if coord(A[i])<coord(B[j]): out.append(A[i]); i+=1
        else if coord(B[j])<coord(A[i]): out.append(B[j]); j+=1
        else:
            value=A[i].value+B[j].value
            if value!=0: out.append((A[i].row,A[i].col,value))
            i+=1; j+=1
```

https://github.com/xinhi6716

Complete sparse addition and matrix-vector multiplication (continued)

```
    append remaining A[i:] and B[j:]; return out
CSR_matvec(R,C,row_ptr,col_idx,values,x):
    require len(x)==C
    y=[0]*R
    for r=0..R-1:
        for p=row_ptr[r]..row_ptr[r+1]-1:
            y[r]+=values[p]*x[col_idx[p]]
    return y
```

Let B contain(0,1,−5),(2,0,3). Adding it to A cancels(0,1), emits A's(1,0,−2),(1,3,7), then B's(2,0,3), then A's(2,1,4). Four entries remain, not kA+kB=6. For x=[1,2,3,4], row0 gives 5×2=10; row1 accumulates−2 then26; row2 gives 4×2=8.

CSR requires 0≤row_ptr[r]≤row_ptr[r+1]≤k, row_ptr[0]=0, row_ptr[R]=k, and valid columns in[0, C). Equal array lengths alone do not establish these invariants. Arithmetic costs assume fixed-range values and do not cover overflow or arbitrary-precision accumulation.

String ADT and encoding levels

- A string is a finite sequence over alphabet Σ. Basic operations include length, concatenation, substring, and comparison.
- `std::string` stores char code units and can contain embedded `\0`. `size()` determines its logical length. C-string termination rules do not always apply.
- A UTF-8 code point can occupy multiple bytes. A user-perceived character can contain multiple code points.
- Matching here compares fixed-size symbols. If symbols are UTF-8 bytes, returned positions are byte positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Copying and viewing strings

- With contiguous code units and a stored length, indexed access can cost O(1). Copying a substring of length k takes Θ(k) time and output space.
- Concatenating lengths N and M into an independent result copies Θ(N+M) units. Repeatedly copying the whole accumulated prefix can give quadratic total work.
- A nonowning string view stores a start and length. Slicing can take constant time, but the view does not extend the source lifetime. A released or reallocated source invalidates the view.
- Lexicographic comparison searches for the first differing symbol. The comparison costs $O(\ell + 1)$, where ℓ is the common-prefix length and the additional term covers the end or mismatch check. Equal hashes, pointers, or lengths do not generally replace content equality.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Naive matching and the KMP prefix function

- Let text length be N and pattern length be M. Comparing at every candidate start uses at most `O(NM)` symbol comparisons; including initialization and empty inputs, time is `O(1+NM)`, for example repeated symbols with a mismatch at the last pattern position.
- Define `π[i]` as the length of the longest proper prefix of `P[0..i]` that is also a suffix.
- After q matched symbols, a mismatch sets `q=π[q-1]` to try a shorter known border. The text position never moves backward.
- Prefix construction and search each have a linear total number of fallback steps. The overall bounds are `O(N+M)` time and `O(M)` auxiliary space.

The fallback chain enumerates possible shorter borders. Any border surviving the mismatch must be a border of the already matched prefix, so skipping other lengths cannot discard a match. Each successful extension increases q by at most one. Each fallback strictly decreases q, so the total number of fallbacks is bounded by prior increases, separately during construction and search.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

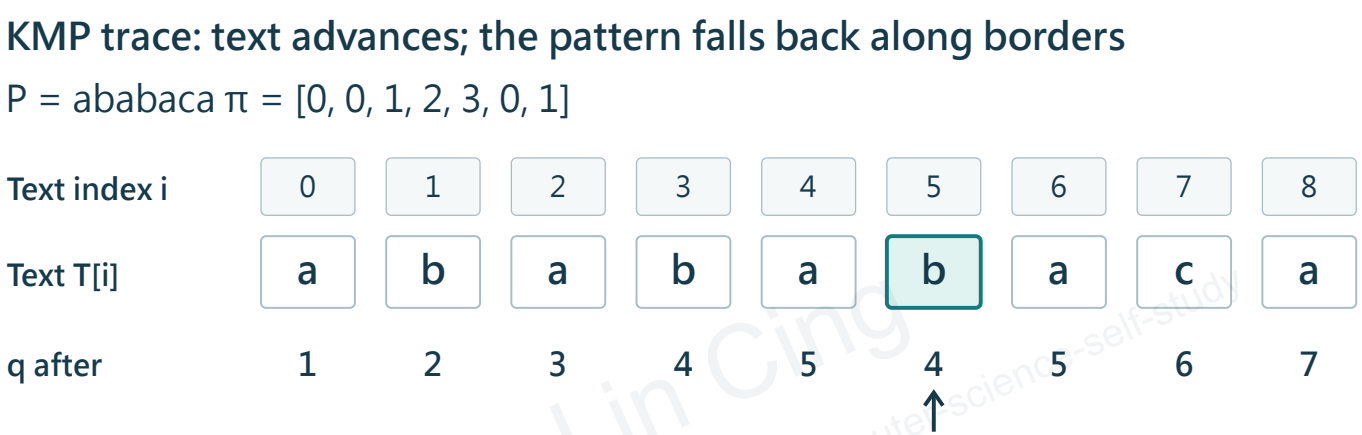
Worked KMP trace

- For `P=ababaca`, the prefix function is `[0,0,1,2,3,0,1]`.
- After matching `ababa`, the next text symbol is b while the pattern expects c. Fall back from `q=5` to `π[4]=3`.
- Now `P[3]=b` matches that same text symbol, giving `q=4` without rescanning earlier text.
- In `abababaca`, the complete match starts at zero-based position 2. Our interface defines an empty pattern to match at position 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP: reuse the same text character after fallback

In the figure, q is the length of the matched pattern prefix.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete KMP construction and search

```
prefix(P):
    pi=[0]*len(P)
    for i=1..len(P)-1:
        q=pi[i-1]
        while q>0 and P[q]!=P[i]: q=pi[q-1]
        if P[q]==P[i]: q+=1
        pi[i]=q
    return pi
first_match(T,P):
    if len(P)==0: return 0
    pi=prefix(P); q=0
    for i=0..len(T)-1:
        while q>0 and P[q]!=T[i]: q=pi[q-1]
        if P[q]==T[i]: q+=1
        if q==len(P): return i-len(P)+1
    return not_found
```

Complete KMP construction and search (continued)

For P=ababaca, construction steps are i=1:b, q=0→0; i=2:a,0→1; i=3:b,1→2; i=4:a,2→3; i=5:c,3→1→0 with no extension; i=6:a,0→1. Thus pi=[0,0,1,2,3,0,1]. Reading T=abababaca gives q=1,2,3,4,5,4,5,6,7. At text index 5, b makes q fall from 5 to 3 before extending to 4. The match ends at i=8 and starts at 2.

Construction accesses P[q] only for nonempty P and $i \geq 1$. Search handles an empty pattern first and immediately returns when q=M, avoiding P[M]. To enumerate overlapping matches, set q=pi[M−1] after each success instead of clearing it. Fixed-symbol comparison gives O(N+M) total work. Byte-oriented UTF-8 matching returns byte offsets, not grapheme indices.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft vector.overview and vector.capacity; Knuth, Morris, Pratt, Fast Pattern Matching in Strings (1977).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study